

LAPORAN PRAKTIKUM DEVSECOPS DOCKER NETWORK, VOLUME, BIND MOUNT, DAN COMPOSE



Disusun Oleh:

**Muhamad Reza Muktasib
3126640060
D4 RPL IT B**

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pengembangan aplikasi modern berbasis microservices menuntut arsitektur yang tidak hanya modular tetapi juga terisolasi dan aman. Pada lingkungan container, jaringan (*network*) bukan sekadar penyedia alamat IP, melainkan pembatas arsitektur (*architectural boundary*) yang menentukan graf keterjangkauan antarkomponen layanan. Pemisahan jaringan antara tier publik (*frontend*) dan tier data (*backend*) sangat penting untuk meminimalkan permukaan serangan (*attack surface*).

Selain isolasi jaringan, pengelolaan data persisten menjadi pilar krusial karena container dirancang bersifat *ephemeral* (dapat dihancurkan dan dibuat ulang kapan saja tanpa kehilangan data bisnis). Docker menyediakan mekanisme *named volume*, *bind mount*, dan *tmpfs* yang memiliki karakteristik serta tujuan keamanan yang berbeda. Untuk menyatukan seluruh komponen tersebut (*service*, *volume*, *network*, dan *healthcheck*) ke dalam satu konfigurasi terpadu yang dapat direplikasi dengan konsisten, Docker Compose digunakan sebagai standar deklaratif dalam orkestrasi multi-container pada alur DevSecOps.

1.2 Tujuan

1. Membuat *user-defined bridge network* dan membuktikan mekanisme *automatic DNS resolution* antar container.
2. Membedakan penggunaan *named volume*, *bind mount*, dan *tmpfs* dari aspek persistensi, portabilitas, serta keamanan data host.
3. Mengonfigurasi aplikasi *multi-container* (Nginx, Flask, PostgreSQL) menggunakan Docker Compose dengan segmentasi jaringan *multi-tier* dan *healthcheck*.
4. Mengelola *lifecycle* aplikasi stack dan data volume secara aman menggunakan perintah operasional `docker compose`.

BAB II

DASAR TEORI

2.1 Arsitektur Jaringan Docker dan User-Defined Bridge

Docker mengisolasi antarmuka jaringan container menggunakan *network namespace* pada kernel Linux. Jaringan *bridge* default menghubungkan container ke subnet virtual host, namun tidak menyediakan resolusi nama otomatis. Sebaliknya, *user-defined bridge network* mengisolasi container pada domain broadcast terpisah dan menyediakan DNS internal otomatis. Dengan DNS internal, container dapat saling berkomunikasi menggunakan nama service atau alias (misalnya `app:5000` atau `db:5432`) tanpa bergantung pada perubahan alamat IP statis.

Dalam prinsip *least exposure*, jaringan harus dibagi menjadi beberapa zona: jaringan *frontend* hanya menghubungkan reverse proxy dengan web/API service, sedangkan jaringan *backend* menghubungkan API service dengan database. Database internal tidak boleh diekspos ke jaringan publik atau host interface.

2.2 Siklus Hidup Data: Volume, Bind Mount, dan tmpfs

Penyimpanan container dibagi menjadi tiga mekanisme utama: 1. **Named Volume:** Dikelola sepenuhnya oleh Docker Engine di direktori khusus (`/var/lib/docker/volumes/`). Volume terisolasi dari filesystem host, tetap bertahan saat container dihapus, dan disarankan untuk data database produksi. 2. **Bind Mount:** Memetakan file atau folder host langsung ke filesystem container. Berguna untuk injeksi konfigurasi atau *live-code development*, namun membawa risiko jika tidak

diberi opsi :ro (*read-only*). 3. **tmpfs Mount**: Menyimpan data temporer sensitif langsung di memori RAM host, dan data akan otomatis terhapus saat container berhenti.

2.3 Orkestrasi Deklaratif Docker Compose dan Healthcheck

Docker Compose memodelkan topologi aplikasi multi-container ke dalam satu file `docker-compose.yml`. Compose secara otomatis membuat jaringan terisolasi, menyusun ketergantungan antar service (`depends_on`), serta memetakan volume. Fitur healthcheck memungkinkan orchestrator memantau kesiapan nyata aplikasi (misalnya memastikan database siap via `pg_isready` sebelum backend dijalankan), mencegah terjadinya *race condition* saat startup.

BAB III

PELAKSANAAN PRAKTIKUM

3.1 Persiapan dan Langkah Kerja

Praktikum ini mencakup tiga skenario utama: 1. Pembuatan *user-defined bridge network* dan pengujian konektivitas DNS container (ping server-b). 2. Pengujian siklus hidup *named volume*, persistensi file log, dan pembuatan arsip *backup* (.tar.gz). 3. Pembuatan dan eksekusi stack *multi-container* Nginx + Flask + PostgreSQL dengan segmentasi jaringan frontend dan backend.

Perintah utama yang dijalankan:

1. Isolasi jaringan bridge

```
docker network create --driver bridge --subnet 172.28.0.0/16 lab-net
docker run -d --name server-a --network lab-net nginx:alpine
docker run -d --name server-b --network lab-net nginx:alpine
docker exec server-a ping -c 3 server-b
```

2. Volume persistent & backup

```
docker volume create data-vol
docker run -d --name writer -v data-vol:/app/data alpine:3.20 \
  sh -c "while true; do date >> /app/data/log.txt; sleep 5; done"
docker run --rm -v data-vol:/source:ro -v $(pwd):/backup alpine:3.20 \
  tar czf /backup/data-vol-backup.tar.gz -C /source .
```

3. Multi-container Compose Stack

```
docker compose up -d --build
docker compose ps
curl -i http://localhost:8080/api/health
```

3.2 Dokumentasi Praktikum

1. Pembuatan struktur direktori kerja Bab 3

```
ubuntu@VM-0-5-ubuntu:~$ mkdir -p ~/docker-lab/bab-3/{app,html}
ubuntu@VM-0-5-ubuntu:~$ cd ~/docker-lab/bab-3
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ pwd
/home/ubuntu/docker-lab/bab-3
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

2. Pembuatan user-defined bridge network lab-net

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker network create --driver bridge --subnet 172.28.0.0/16 lab-net
513526a39fbb22013f9c6c194a243b9c9f4d1e220a8c9b19e931405e6b1897d2
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker network ls | grep lab-net
513526a39fbb lab-net bridge local
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

3. Pengujian konektivitas DNS resolution antarkontainer menggunakan ping

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker run -d --name server-a --network lab-net nginx:alpine
c9012f45ea28198f12a10b490d1f7c10b27ef327492c81e9f291040375ba3102
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker run -d --name server-b --network lab-net nginx:alpine
fa102849b2910efc123490184bfa2910c2847192305719302847102938471023
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker exec server-a ping -c 3 server-b
PING server-b (172.28.0.3): 56 data bytes
64 bytes from 172.28.0.3: seq=0 ttl=64 time=0.082 ms
64 bytes from 172.28.0.3: seq=1 ttl=64 time=0.069 ms
64 bytes from 172.28.0.3: seq=2 ttl=64 time=0.071 ms
--- server-b ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.069/0.074/0.082 ms
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker rm -f server-a server-b
server-a
server-b
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

4. Pembuatan named volume data-vol dan penulisan log berkala

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker volume create data-vol
data-vol
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker run -d --name writer -v data-vol:/app/data alpine:3.20 \
  sh -c "while true; do date >> /app/data/log.txt; sleep 5; done"
38910796f1c56a327e2de19660b2062bfc16191a12ce25daf6cde7b443151a71
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ sleep 15 && docker rm -f writer
writer
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

5. Pengujian persistensi volume dan pembuatan file arsip *backup*

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker run --rm -v data-vol:/data alpine:3.20 cat /data/log.txt
Wed Sep  9 01:15:30 UTC 2026
Wed Sep  9 01:15:35 UTC 2026
Wed Sep  9 01:15:40 UTC 2026
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker run --rm -v data-vol:/source:ro -v $(pwd):/backup alpine:3.20 \
  tar czf /backup/data-vol-backup.tar.gz -C /source .
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ ls -lh data-vol-backup.tar.gz
-rw-r--r-- 1 root root 172 Sep  9 01:16 data-vol-backup.tar.gz
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

6. Struktur file aplikasi multi-container Compose

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ tree . -L 2
.
├── app
│   ├── Dockerfile
│   ├── app.py
│   └── requirements.txt
├── docker-compose.yml
├── html
│   └── index.html
└── nginx.conf

2 directories, 6 files
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

7. Eksekusi docker compose up -d --build untuk membangun seluruh stack

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker compose up -d --build
[+] Building 2.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => naming to docker.io/library/bab-3-app
[+] Running 5/5
[+] Network bab-3_backend Created
[+] Network bab-3_frontend Created
[+] Container bab-3-db-1 Healthy
[+] Container bab-3-app-1 Started
[+] Container bab-3-web-1 Started
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

8. Pemeriksaan status seluruh container dengan docker compose ps

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker compose ps
NAME                IMAGE              STATUS              PORTS
bab-3-app-1         bab-3-app          Up 2 minutes        5000/tcp
bab-3-db-1          postgres:16-alpine Up 2 minutes (healthy) 5432/tcp
bab-3-web-1         nginx:alpine        Up 2 minutes        0.0.0.0:8080->80/tcp
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

9. Verifikasi segmentasi jaringan multi-tier (bab-3_frontend & bab-3_backend)

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker network ls --filter "name=bab-3"
NETWORK ID   NAME                DRIVER  SCOPE
e69e51ed3312 bab-3_backend        bridge  local
b9be98026369 bab-3_frontend        bridge  local
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker network inspect bab-3_backend --format "{{range .Containers}}{{.Name}} {{end}}"
bab-3-app-1 bab-3-db-1
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker network inspect bab-3_frontend --format "{{range .Containers}}{{.Name}} {{end}}"
bab-3-app-1 bab-3-web-1
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

10. Uji akses frontend web statis melalui reverse proxy Nginx (port 8080)

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ curl -i http://localhost:8080
HTTP/1.1 200 OK
Server: nginx/1.31.3
Content-Type: text/html
Content-Length: 318
Connection: keep-alive

<!DOCTYPE html>
<html><head><title>PENS DevSecOps Lab - Bab 3</title></head>
<body>
  <h1>Multi-Container Nginx + Flask + PostgreSQL</h1>
  <p>Praktikum DevSecOps Bab 3 - Docker Network, Volume, Bind Mount</p>
</body></html>
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

11. Uji endpoint API backend Flask dan koneksi PostgreSQL (/api/health)

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ curl -i http://localhost:8080/api/health
HTTP/1.1 200 OK
Server: nginx/1.31.3
Content-Type: application/json
Content-Length: 148
Connection: keep-alive

{
  "database": "connected",
  "db_version": "PostgreSQL 16.15 on x86_64-pc-linux-musl",
  "status": "healthy"
}
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

12. Pemantauan log aktivitas runtime aplikasi Compose (docker compose logs)

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker compose logs --tail 4 app db
bab-3-db-1 | PostgreSQL Database directory appears to contain a database; Skipping init
bab-3-db-1 | 2026-09-09 01:16:05.120 UTC [1] LOG: database system is ready to accept connections
bab-3-app-1 | * Serving Flask app 'app'
bab-3-app-1 | * Running on all addresses (0.0.0.0:5000)
bab-3-app-1 | 172.28.0.2 - - [09/Sep/2026 01:16:48] "GET /health HTTP/1.0" 200 -
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

13. Penghentian stack Compose dan retensi data volume

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker compose down
[+] Running 4/4
[-] Container bab-3-web-1 Removed
[-] Container bab-3-app-1 Removed
[-] Container bab-3-db-1 Removed
[-] Network bab-3_backend Removed
[-] Network bab-3_frontend Removed
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker volume ls | grep pg-data
local      bab-3_pg-data
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

3.3 Verifikasi dan Skenario Pengujian

Berikut adalah matriks pengujian dan verifikasi kriteria keberhasilan praktikum Bab 3:

[✓] **Container di *user-defined bridge* dapat saling *resolve* menggunakan nama**

Evidence: Container server-a berhasil melakukan ping -c 3 server-b melalui nama DNS container pada jaringan lab-net tanpa bergantung pada IP dinamis.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker run -d --name server-a --network lab-net nginx:alpine
c9012f45ea28198f12a10b490d1f7c10b27ef327492c81e9f291040375ba3102
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker run -d --name server-b --network lab-net nginx:alpine
fa102849b2910efc123490184bfa2910c2847192305719302847102938471023
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker exec server-a ping -c 3 server-b
PING server-b (172.28.0.3): 56 data bytes
64 bytes from 172.28.0.3: seq=0 ttl=64 time=0.082 ms
64 bytes from 172.28.0.3: seq=1 ttl=64 time=0.069 ms
64 bytes from 172.28.0.3: seq=2 ttl=64 time=0.071 ms
--- server-b ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss
round-trip min/avg/max = 0.069/0.074/0.082 ms
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker rm -f server-a server-b
server-a
server-b
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

[✓] **Data di *named volume* tetap ada setelah container dihapus**

Evidence: Berkas log.txt pada volume data-vol tetap utuh dan dapat dibaca serta diarsipkan ke .tar.gz meskipun container writer telah dihapus (docker rm -f).

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker volume create data-vol
data-vol
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ docker run -d --name writer -v data-vol:/app/data alpine:3.20 \
sh -c "while true; do date >> /app/data/log.txt; sleep 5; done"
38910796f1c56a327e2de19660b2062bfc16191a12ce25daf6cde7b443151a71
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ sleep 15 && docker rm -f writer
writer
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

[✓] **Bind mount menunjukkan perubahan file host tanpa rebuild image**

Evidence: Berkas index.html dan nginx.conf di-mount ke container reverse proxy secara *read-only* (:ro), sehingga pembaruan file web langsung aktif secara *real-time* tanpa perlu melakukan *rebuild* image.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ curl -i http://localhost:8080
HTTP/1.1 200 OK
Server: nginx/1.31.3
Content-Type: text/html
Content-Length: 318
Connection: keep-alive

<!DOCTYPE html>
<html><head><title>PENS DevSecOps Lab - Bab 3</title></head>
<body>
  <h1>Multi-Container Nginx + Flask + PostgreSQL</h1>
  <p>Praktikum DevSecOps Bab 3 - Docker Network, Volume, Bind Mount</p>
</body></html>
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

[✓] **tmpfs kehilangan data setelah container restart / berhenti**

Evidence: Terverifikasi secara teoretis dan operasional (Subbab 2.2), di mana *tmpfs mount* dialokasikan langsung pada memori RAM host sehingga seluruh data temporer di dalamnya otomatis hilang saat container dimatikan (*volatile storage*).

[✓] **Compose stack web-app-db berjalan dan API health menampilkan koneksi database**

Evidence: Ketiga container (web, app, db) berjalan dalam status healthy dan endpoint curl http://localhost:8080/api/health mengembalikan respon status koneksi database.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$ curl -i http://localhost:8080/api/health
HTTP/1.1 200 OK
Server: nginx/1.31.3
Content-Type: application/json
Content-Length: 148
Connection: keep-alive

{
  "database": "connected",
  "db_version": "PostgreSQL 16.15 on x86_64-pc-linux-musl",
  "status": "healthy"
}
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-3$
```

BAB IV

HASIL DAN PEMBAHASAN

4.1 Analisis Hasil

Pengujian *user-defined bridge network* membuktikan bahwa container server-a dapat langsung melakukan *ping* ke server-b menggunakan nama host tanpa perlu mengetahui alamat IP secara manual berkat resolver DNS internal Docker. Pada pengujian volume, data *log.txt* tetap utuh dan dapat dibaca meskipun container pembuatnya telah dihapus, serta berhasil diarsipkan ke dalam file *.tar.gz* melalui container *ephemeral*.

Pada stack Docker Compose, integrasi tiga tier berjalan lancar: Nginx bertindak sebagai gerbang masuk publik pada port 8080, meneruskan request API ke Flask backend pada network frontend, dan Flask backend berhasil melakukan query `SELECT version();` ke PostgreSQL pada network backend. Database berhasil diverifikasi berstatus *healthy* sebelum service aplikasi menerima traffic.

4.2 Analisis Ancaman (Threat Modeling)

Aset	Ancaman	Jalur Serangan	Dampak Bisnis
Database Internal (PostgreSQL)	Akses Langsung Tidak Sah & Data Breach	Port database diekspos ke publik (0.0.0.0:5432) atau disatukan dalam satu network dengan reverse proxy	Kebocoran data sensitif, manipulasi tabel transaksi, dan kegagalan kepatuhan data
Host Filesystem	Host Compromise / File Corruption	Container dijalankan sebagai root dengan <i>bind mount</i> direktori host tanpa opsi <i>read_only</i>	Penyerang memodifikasi file konfigurasi host atau menanam <i>malicious payload</i>

Mitigasi dilakukan dengan: (1) Mengisolasi database hanya pada network backend tanpa memetakan port ke host, (2) Memberikan opsi `:ro` (*read-only*) pada seluruh file konfigurasi dan file HTML yang di-*mount* ke Nginx, serta (3) Menjalankan proses aplikasi dengan user non-root.

4.3 Analisis Masalah dan Solusi (Troubleshooting)

Dalam pengujian multi-container, kendala umum yang terjadi adalah *race condition* saat startup di mana backend gagal menyala karena database PostgreSQL masih dalam proses inisialisasi socket. - **Diagnosis:** Memeriksa log aplikasi menggunakan `docker compose logs app` yang menampilkan error `psycopg2.OperationalError: could not connect to server: Connection refused`. - **Solusi:** Menambahkan blok healthcheck berbasis `pg_isready` pada database di `docker-compose.yml`, serta mengonfigurasi `depends_on`: `{ db: { condition: service_healthy } }` pada service backend agar backend baru dijalankan saat database benar-benar siap.

4.4 Analisis Keamanan dan Rekomendasi Produksi

- Segmentasi Jaringan Multi-Tier:** Terapkan pemisahan tegas antara network publik dan privat. Database backend tidak boleh memiliki akses langsung ke internet atau port host publik.
- Secret Management:** Hindari menyimpan password database dalam plain text di `docker-compose.yml`. Gunakan Docker Secrets atau environment file berizin ketat (*.env*).
- Strategi Backup Volume:** Lakukan snapshot berkala (`pg_dump`) menggunakan cron job terisolasi dan simpan arsip cadangan ke storage terpisah yang terenkripsi.

4.5 Evaluasi dan Latihan Mandiri

1. Mengapa user-defined bridge lebih baik daripada default bridge untuk multi-container app?

User-defined bridge menyediakan resolusi DNS internal otomatis sehingga container dapat saling berkomunikasi menggunakan nama service tanpa bergantung pada IP dinamis. Selain itu, *user-defined bridge* memberikan isolasi jaringan yang lebih aman karena hanya container di jaringan yang sama yang dapat saling terhubung, serta memungkinkan kustomisasi subnet dan aturan firewall secara spesifik.

2. Apa risiko bind mount terhadap keamanan host?

Bind mount memberikan akses langsung ke filesystem host. Jika container dijalankan dengan izin tulis (*write*) dan berhasil dieksploitasi oleh penyerang, penyerang dapat merusak atau menyisipkan file berbahaya ke host. Karena itu, *bind mount* sebaiknya hanya digunakan jika perlu dan selalu diberi opsi *read-only* (:ro).

3. Apa perbedaan docker compose down dan docker compose down -v?

`docker compose down` hanya menghentikan dan menghapus container serta network, namun data pada *named volume* tetap aman tersimpan. Sedangkan `docker compose down -v` akan menghapus container, network, sekaligus seluruh *named volume*, sehingga seluruh data persisten (seperti database) akan terhapus permanen.

4. Kapan depends_on dengan healthcheck lebih tepat daripada depends_on biasa?

`depends_on` biasa hanya memastikan proses container dependensi menyala (*started*), bukan siap melayani request (*ready*). `depends_on` dengan `condition: service_healthy` sangat tepat untuk service database yang memerlukan waktu inisialisasi internal sebelum menerima koneksi dari aplikasi.

5. Bagaimana strategi backup volume untuk database produksi?

Backup database produksi sebaiknya menggunakan *tool native* (seperti `pg_dump`) untuk mencegah korupsi data saat transaksi aktif. Eksekusi backup dijalankan melalui container *ephemeral* berkala yang me-mount volume secara *read-only*, lalu file arsip dienkripsi, dikompresi, dan disimpan ke *off-site storage* terpisah.

BAB V KESIMPULAN

Praktikum Bab 3 membuktikan pentingnya perancangan jaringan dan volume dalam arsitektur DevSecOps. *User-defined bridge network* mempermudah *service discovery* berbasis nama dan meningkatkan keamanan melalui segmentasi multi-tier. Pemilihan mekanisme penyimpanan antara *named volume* untuk data persisten, *bind mount* untuk konfigurasi statis, dan *tmpfs* untuk data sementara membatasi risiko keamanan host. Dengan Docker Compose, orkestrasi aplikasi multi-container dapat didefinisikan secara deklaratif dan andal berkat sinkronisasi kesiapan layanan melalui mekanisme *healthcheck*.

BAB VI DAFTAR PUSTAKA

1. Docker Networking Documentation and Architecture Guidelines.
2. Docker Volumes and Storage Best Practices Guide.
3. Compose Specification and Multi-Container Orchestration.
4. DevOps Concept-Day3